VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DEEP LEARNING AND ITS APPLICATIONS
(CO3133)

Assignment 2 Report

Vietnamese Food Classification with CNN and ResNet-50 on 30VNFoods

---

Instructor: Lê Thành Sách
Group: NNs
Project team
Nguyễn Nhật Nam – 2352778
Hồ Hồng Phúc Nguyên – 2352824
Phan Quốc Đại Sơn – 2353053

HO CHI MINH CITY, NOVEMBER 2026

Abstract

We classify photographs into 30 Vietnamese dish categories in 30VNFoods. We use exact and perceptual hashes to filter duplicates and label conflicts, then compare a CNN trained from scratch with two ImageNet-pretrained ResNet-50 variants. One variant trains only the classifier head, and the other fine-tunes the entire network. Checkpoints are selected by validation macro-F1 and evaluated on the same 4,984 cleaned test images.

Across three matched seeds, full fine-tuning reaches 89.08% mean accuracy and 0.8922 mean macro-F1, compared with 76.79% and 0.7720 for the frozen backbone. The mean paired gain is 0.1202 macro-F1. The single CNN run reaches 54.51% accuracy and 0.5403 macro-F1. Full fine-tuning takes about twice as long to train as the frozen variant, while both average about 4.3 ms per image in the batched forward pass. In the validation-selected full checkpoint, 22 of 191 Banh canh images are still classified as Hu tieu.

Keywords. Vietnamese food classification, 30VNFoods, transfer learning, ResNet-50, error analysis.

Contents

Abstract **i**

I Problem and Data Description 1

1 Problem Formulation 1
1.1 Motivation . . . 1
1.2 Input and Output . . . 1
1.3 Research Question . . . 1

2 Dataset Description 1
2.1 Source and Annotations . . . 1
2.2 Exploratory Data Analysis . . . 2

3 Data Preparation 4
3.1 Duplicate Detection and Split Construction . . . 4
3.2 Image Transforms and Batching . . . 5

II Methodology 6

4 Overall Pipeline 6

5 Models 7
5.1 CNN Baseline . . . 7
5.2 ImageNet-Pretrained ResNet-50 . . . 8

6 Loss and Optimization 9
6.1 Multiclass Cross-Entropy . . . 9
6.2 Adam and Learning Rates . . . 9
6.3 Regularization . . . 9

7 Training and Evaluation Protocol 10
7.1 Training Setup . . . 10
7.2 Hyperparameter Selection and Configuration Development . . . 10
7.3 Hardware and Software . . . 10
7.4 Checkpoint Selection and Early Stopping . . . 11
7.5 Evaluation Metrics . . . 11
7.6 Final Test and Compute Protocol . . . 11

III Implementation Results **12**

8 Training Behaviour **12**
8.1 CNN Baseline . . . 12
8.2 Paired ResNet-50 Runs . . . 13
8.3 Training Time and Implementation . . . 13

9 Quantitative Results **14**
9.1 Cleaned Test Performance . . . 14
9.2 Compute Cost . . . 14
9.3 Per-class F1 . . . 15
9.4 Confusion Matrix . . . 16

10 Qualitative Results **17**
10.1 Correct, Difficult, and Failed Predictions . . . 17
10.2 Changes After Full Fine-tuning . . . 18
10.3 Grad-CAM for the Banh Canh Failure . . . 18

11 Comparison and Discussion **19**
11.1 Effect of Backbone Adaptation . . . 19
11.2 Where the Predictions Change . . . 19
11.3 Accuracy and Compute Trade-offs . . . 19

12 Error Analysis **20**
12.1 Error Groups and Examples . . . 20
12.2 Checks of the EDA Hypotheses . . . 21

13 Ablation Study **22**
13.1 Controlled Factor and Hypothesis . . . 22
13.2 Paired Test Results . . . 22

14 Limitations **23**

15 Conclusion **23**

16 Reproducibility **24**
16.1 Environment, Dataset, and Checkpoints . . . 24
16.2 Rerun Evaluation and Error Analysis . . . 25
16.3 Rebuild Checkpoints from Training . . . 25

AI Usage Disclosure **26**

References **26**

A Dataset Proposal and Approval **27**

B Supplementary Evidence **27**

C Team Contributions **27**

Part I
Problem and Data Description

1 Problem Formulation

1.1 Motivation

Recognizing Vietnamese dishes from photos can support image search and food catalog organization. The task is challenging because distinct dishes can share broth, toppings, or wrappers, while photos of the same dish vary in preparation, presentation, lighting, and framing. Garnishes or tight crops can obscure informative features such as noodle width or the shape of a wrapped rice cake.

1.2 Input and Output

The prediction unit is one RGB photograph with one of 30 dish labels. Let $x \in \mathbb{R}^{3\times224\times224}$ be the preprocessed image, $y \in \{0, \ldots, 29\}$ its folder-derived class ID, and f_θ a classifier with parameters θ. The classifier returns one logit per class and predicts the class with the largest logit:

$$z = f_\theta(x) \in \mathbb{R}^{30}, \qquad \hat{y} = \underset{k\in\{0,\ldots,29\}}{\arg\max}\ z_k. \tag{1}$$

1.3 Research Question

How much does adapting an ImageNet-pretrained ResNet-50 backbone improve accuracy and macro-F1 over training only its classification head, and what is the associated compute cost? The controlled comparison uses these two ResNet-50 variants. A CNN trained from scratch provides a simpler baseline on the same data partitions. The experiments assess recognition within the 30 classes represented in 30VNFoods.

2 Dataset Description

2.1 Source and Annotations

We use 30VNFoods, the Vietnamese Foods dataset published as `quandang/vietnamese-foods` on Kaggle, pinned to version 11. The release contains 25,136 web-sourced RGB JPEG images in 30 dish categories. The class folder supplies one label per image. The release also supplies lists of source URLs by class. The downloaded ZIP is approximately 4.48 GB and the extracted JPEGs total 4.51 GB, using decimal units. The release includes an MIT license file credited to Quan Dang (2021). The photos originate from third-party websites. The collection and labeling workflow is documented in the authors' project repository.

The cleaned training partition has 17,164 images in 30 classes, exceeding the handbook's image-classification minimums of 5,000 training samples and five classes. The smallest category, Banh pia, retains 306 training, 44 validation, and 89 test images. The dataset proposal and its approval record are listed in Appendix A.

2.2 Exploratory Data Analysis

2.2.1 Class Distribution and Imbalance

Figure 1 shows the original partitions. In the original training set, Bun bo Hue has 1,071 images and Banh pia has 310, a largest-to-smallest ratio of 3.45. After duplicate cleaning, these counts are 1,043 and 306, respectively, giving a ratio of 3.41. Accuracy weights examples equally, so larger classes influence it more. We use macro-F1 as the primary metric to give each of the 30 classes equal weight and report accuracy alongside it.

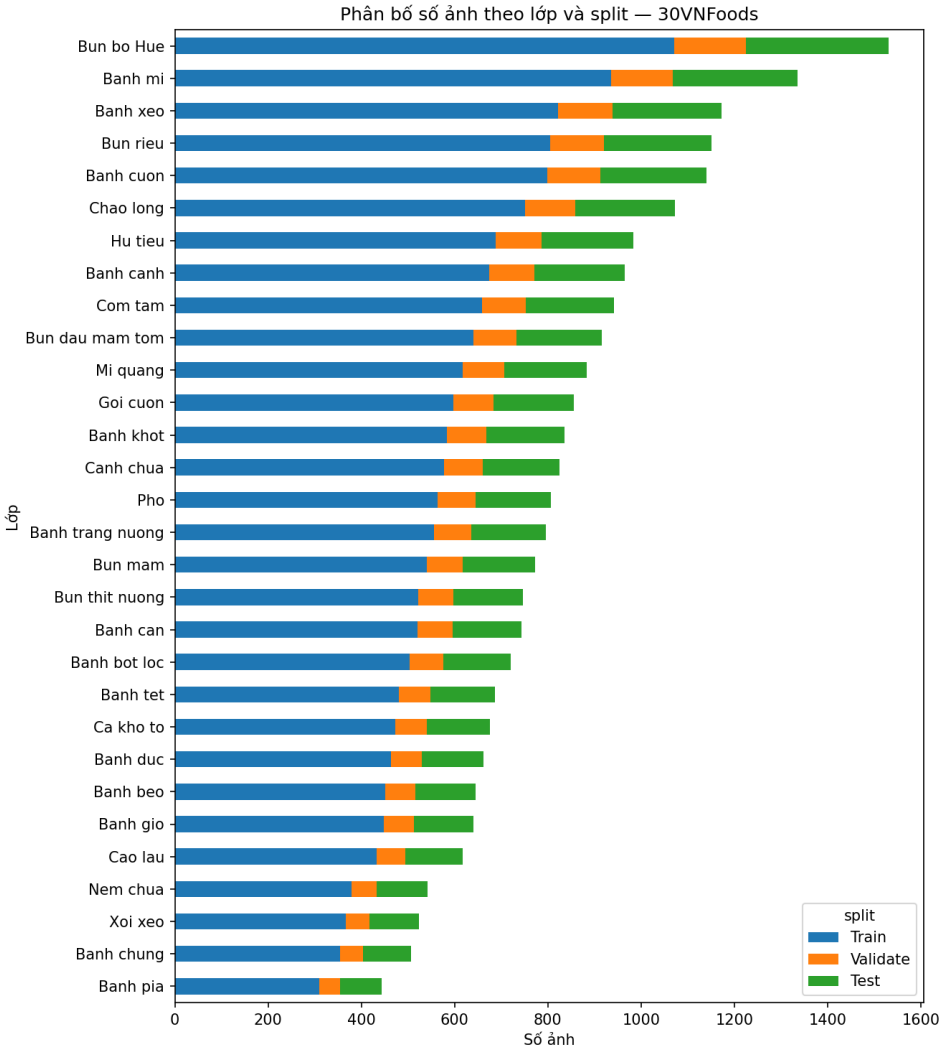


Figure 1: Class counts in the original 30VNFoods train, validation, and test partitions before duplicate removal.

2.2.2 Image Dimensions and Appearance

In the original training set of 17,581 images, median width and height were 1,000 and 720 pixels, respectively. Widths ranged from 100 to 7,360 pixels and heights from 83 to 5,184 pixels. The median width-to-height ratio was 1.33. There were 4,327 square images (24.6%), 193 images (1.1%) with a ratio below 0.5 or above 2, and 251 images (1.4%) with a short side below 224 pixels. Figure 2 shows the ratio distribution.

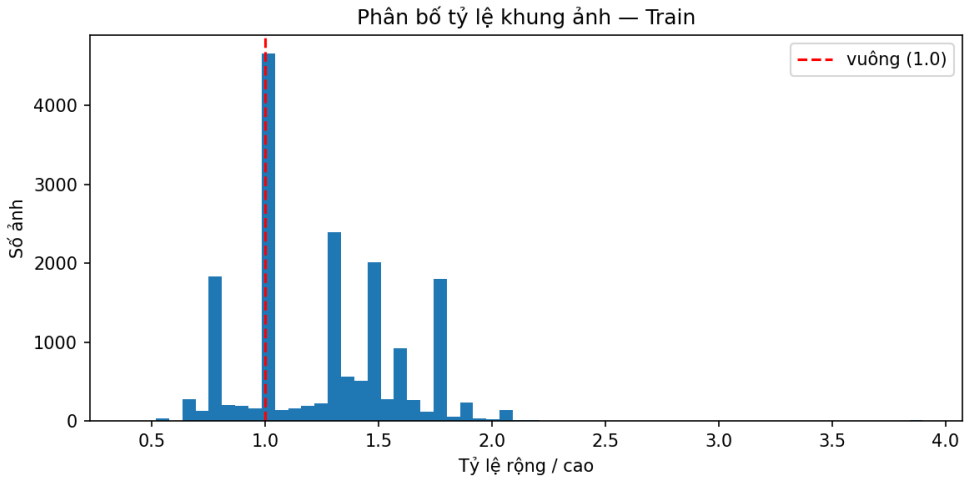


Figure 2: Width-to-height ratios in the original training partition. The dashed line marks square inputs.

In a fixed-seed sample of 3,000 original training images, class distributions of mean RGB intensity overlapped. Banh pia had the highest median intensity in that sample. Visual inspection found examples showing pastries with packaging on white backgrounds. Photos also show that wrapped Banh chung and Banh tet can lose their shape cues in close-up, and that toppings can hide the noodles in visually similar soup dishes. The underlying plots and examples are saved in `notebooks/a2_eda.ipynb`.

These observations motivate four error hypotheses: confusion between Banh chung and Banh tet when the whole shape is hard to see (H1), confusion among Bun bo Hue, Bun rieu, and Banh canh (H2), elevated errors on extreme-aspect images after cropping (H3), and possible reliance on packaging or white backgrounds for Banh pia (H4). Web collection can overrepresent particular sources and presentations. The retained images may also contain label noise beyond the conflicts revealed by duplicate detection.

3 Data Preparation

3.1 Duplicate Detection and Split Construction

We retained the original train, validation, and test assignments, which allocate roughly 70%, 10%, and 20% of each class, and filtered duplicates within and across them. The notebook identified 63 groups of byte-identical files with MD5, including 40 groups that crossed split boundaries. It then compared 64-bit perceptual hashes to find visually similar copies. Images were linked at Hamming distance at most 6, with distance 8 also accepted for pairs sharing a label. The notebook's inspection found more cross-label candidates at distance 8, motivating the tighter threshold for those pairs. The resulting graph contained 427 connected components with at least two images, of which 222 crossed splits and 56 contained conflicting labels.

The connected component is the leakage-control unit. For components with conflicting labels, we dropped all images (118 removals). For single-label components, we retained one image, prioritizing test over validation over training, with filename as the tie-breaker. This removed 196 cross-split copies and 193 additional copies within a split. The notebook opened RGB images in all partitions, and the drop list records duplicate and label-conflict removals. Different photos from the same source website may remain in different partitions because source-domain grouping was not verified.

Table 1: Original and cleaned 30VNFoods partitions. All 30 classes remain in each split.

Partition	Original	Removed	Cleaned v2
Training	17,581	417	17,164
Validation	2,515	34	2,481
Test	5,040	56	4,984
Total	25,136	507	24,629

The shared manifest, `results/a2/splits/split_v2.json`, fixes the image paths, class IDs, and file checksums for every run. The exclusions are listed in `results/a2/tables/a2_drop_list.csv`. Training seeds affect model training and sampling order. The manifest determines partition membership. The validation set supplies the early-stopping and checkpoint-selection scores, and the cleaned test set is reserved for final evaluation.

3.2 Image Transforms and Batching

The dataset loader decodes images as RGB. During training it applies a random resized crop to 224×224 with crop area scale $[0.08, 1.0]$, a random horizontal flip, and color jitter of 0.1 for brightness, contrast, and saturation. During validation and test, it resizes the shorter side to 256 pixels while preserving aspect ratio, then center crops to 224×224 without random augmentation.

Images are converted to float tensors in $[0, 1]$ before channel-wise normalization. For the CNN, the mean and standard deviation come from the cleaned training images resized to 224×224. Both ResNet-50 variants use the ImageNet statistics associated with their pretrained inputs. Table 2 gives the configured values in RGB order. The loader shuffles training samples before batching and returns a tensor of shape $B \times 3 \times 224 \times 224$ and a length-B vector of integer class IDs.

Table 2: Channel-wise normalization statistics used by each model family.

Model family	RGB mean	RGB standard deviation
CNN	$(0.5819, 0.5140, 0.3997)$	$(0.2693, 0.2645, 0.2933)$
ResNet-50	$(0.485, 0.456, 0.406)$	$(0.229, 0.224, 0.225)$

Figure 3 shows augmented crops with their dataset labels. Normalization is reversed for display.



Figure 3: A training batch after preprocessing. The input tensor has shape $16 \times 3 \times 224 \times 224$ and the label vector has length 16.

Part II
Methodology

4 Overall Pipeline

The scratch CNN and both ResNet-50 variants use the cleaned split manifest described in Section 3. Each loader produces batches of RGB images of shape $[B, 3, 224, 224]$, normalized for the selected model family. Each model returns logits of shape $[B, 30]$. Figure 4 shows how a training run produces the files consumed by the separate test evaluator.

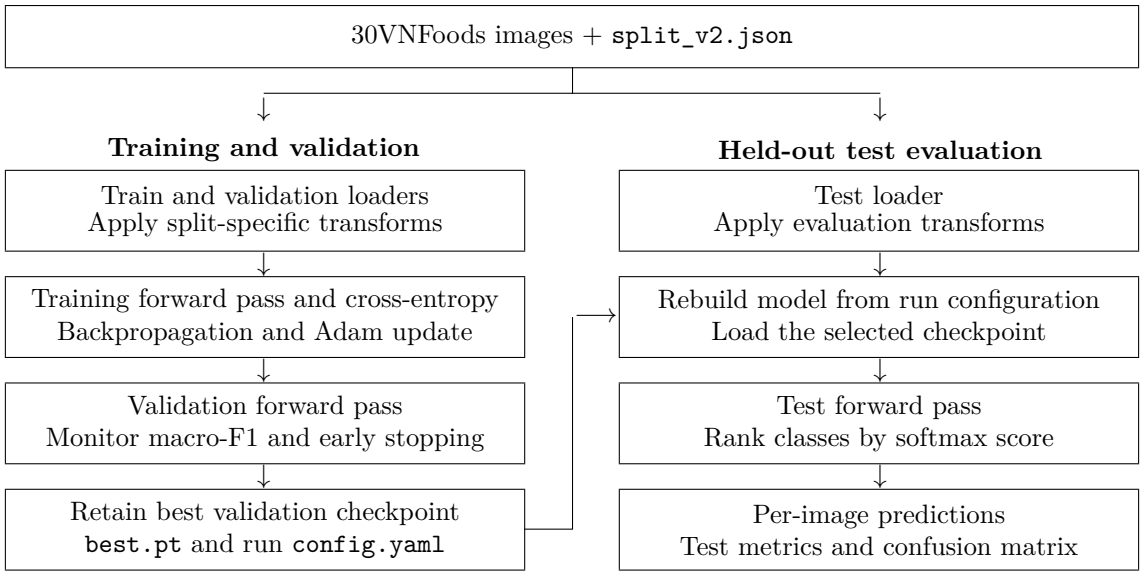


Figure 4: Shared A2 pipeline. The training and validation loop selects the checkpoint before held-out test evaluation.

Training and checkpoint selection. The training entry point `src/a2/train.py` saves the resolved model settings and seed in the run's `config.yaml`. For each training batch, cross-entropy compares the logits with the integer labels and Adam updates the trainable parameters. After each epoch, the validation loop runs in evaluation mode with gradients disabled. The trainer saves `best.pt` when validation macro-F1 strictly improves and stops when patience or the epoch budget is exhausted.

Test inference and post-processing. The evaluator in `src/a2/evaluate.py` reconstructs each model from its saved run configuration, restores `best.pt`, and runs on the cleaned test split. Softmax scores give the top-1 label, its confidence, and the next two ranked classes. It exports one prediction row per image along with accuracy, macro-F1, per-class scores, and a confusion matrix. The test results are used for model comparison and error analysis. Checkpoint selection and early stopping use validation scores.

Implementation. The project supplies the manifest-driven dataset, configuration and checkpoint logic, model wrappers, and evaluation exports. PyTorch supplies batching, neural network layers, differentiation, and Adam. Torchvision supplies image transforms and the pretrained ResNet-50.

5 Models

All three configurations receive the normalized RGB input described in Section 3 and return 30 logits. The models are defined in `src/a2/models.py`. The layer dimensions and parameter counts below follow the configurations saved with the evaluated runs.

5.1 CNN Baseline

The group implements `SmallCNN` with standard PyTorch layers and trains it from scratch. Its feature extractor has four blocks with 32, 64, 128, and 256 output channels. Each block applies a 3×3 convolution with stride 1 and padding 1, BatchNorm, ReLU, and 2×2 max pooling with stride 2. The convolution preserves spatial dimensions and the pooling operation halves them.

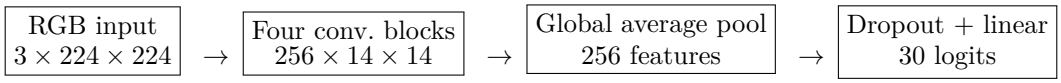


Figure 5: CNN feature extraction and classification. Shapes exclude the batch dimension.

Global average pooling summarizes each final feature map as a single value. The resulting 256 features pass through dropout with probability 0.3 during training, then a linear layer with $256 \times 30 + 30 = 7{,}710$ parameters. Table 3 details the feature dimensions and the total of **397,086 trainable parameters**. Weight sharing and global pooling keep the baseline compact. Repeated downsampling and spatial averaging can weaken cues that depend on fine structure or location.

Table 3: CNN output shapes and parameter counts. Block counts include convolution weights and biases and BatchNorm scale and shift.

Stage	Output per image	Parameters
Block 1	$32 \times 112 \times 112$	960
Block 2	$64 \times 56 \times 56$	18,624
Block 3	$128 \times 28 \times 28$	74,112
Block 4	$256 \times 14 \times 14$	295,680
Global average pool and flatten	256	0
Dropout and linear classifier	30	7,710
Total		397,086

5.2 ImageNet-Pretrained ResNet-50

Both transfer-learning configurations use Torchvision's implementation of He et al.'s ResNet-50, initialized with `IMAGENET1K_V2` weights. The convolutional stem feeds four stages with 3, 4, 6, and 3 residual bottleneck blocks. Each bottleneck applies a 1×1, 3×3, and 1×1 convolution along the residual path and adds the shortcut. For a 224×224 input, the last stage outputs a $2048 \times 7 \times 7$ tensor. Global average pooling yields 2,048 features. The group replaces the ImageNet classifier with a newly initialized 30-way linear head containing $2048 \times 30 + 30 = 61{,}470$ parameters.

Frozen backbone. The wrapper disables gradients for the backbone, and the optimizer receives only the classifier parameters. During training, the backbone stays in evaluation mode so its BatchNorm running means and variances retain the pretrained values. The linear head learns to classify the fixed features. This reduces training cost, while the representation stays fixed when ImageNet features differ from those needed for the dish labels.

Full fine-tuning. The optimizer receives both the backbone and the head. Backbone parameters receive gradients, and BatchNorm updates its running statistics on training batches. This allows the representation to adapt to the dataset and adds the cost of backpropagation through the backbone. Both ResNet variants execute the same architecture at inference with their respective learned states.

Table 4: Model parameters in the reported runs. Total parameters include frozen weights and biases. BatchNorm running statistics are buffers.

Model	Total parameters	Trainable parameters	Updated modules
CNN baseline	397,086	397,086	Entire CNN
ResNet-50 frozen	23,569,502	61,470	Classifier head
ResNet-50 full	23,569,502	23,569,502	Backbone and head

6 Loss and Optimization

The seven evaluated runs share the same loss and optimizer. The CNN run uses the training engine inherited from Assignment 1. The paired ResNet-50 runs use the A2 optimizer in `src/a2/trainer.py` to assign separate learning rates to the classifier and trainable backbone. The settings below come from the resolved `config.yaml` saved with each run.

6.1 Multiclass Cross-Entropy

For a batch of B images, let $z \in \mathbb{R}^{B\times 30}$ be the model logits and $y_i \in \{0, \ldots, 29\}$ the class index for image i. The batch objective is

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(z_{i,y_i})}{\sum_{k=0}^{29}\exp(z_{i,k})}. \tag{2}$$

PyTorch's `CrossEntropyLoss` receives the raw logits and integer labels and uses mean reduction with equal sample weights and zero label smoothing. For the reported epoch loss, the engine weights each batch mean by its actual number of images. Cross-entropy drives parameter updates. Validation macro-F1, which gives each class equal weight, drives checkpoint selection.

6.2 Adam and Learning Rates

Every run uses `torch.optim.Adam` with the default moment coefficients $(0.9, 0.999)$, $\epsilon = 10^{-8}$, and weight decay 10^{-4}. This is coupled L2 regularization on the optimized parameters. Adam uses estimates of the first and second moments of the gradients to scale the updates. Each parameter group's configured learning rate stays fixed throughout training.

Table 5: Learning rates by optimizer group in the evaluated runs.

Model	Parameter group in Adam	Learning rate
CNN baseline	All CNN parameters	10^{-3}
ResNet-50 frozen	Classifier head	10^{-3}
ResNet-50 full	Classifier head	10^{-3}
	Backbone	10^{-4}

The frozen configuration stores `lr_backbone = 0` and excludes the backbone from the optimizer. Full fine-tuning keeps the same head learning rate and uses one tenth of that rate for the pretrained backbone to allow smaller updates to the transferred features.

6.3 Regularization

All runs use the training augmentation described in Section 3. The CNN applies dropout with probability 0.3 before its classifier. The two ResNet-50 variants use a single linear classifier, with backbone BatchNorm behavior as described in Section 5.2. The trainer uses validation macro-F1 for early stopping within each run's epoch budget.

7 Training and Evaluation Protocol

This protocol covers the seven checkpoints evaluated on the fixed split of 17,164 training, 2,481 validation, and 4,984 test images. Settings were checked against saved configurations, epoch histories, and evaluation records.

7.1 Training Setup

Table 6 gives the resolved settings saved with each run. Section 6 gives the optimization settings.

Table 6: Training settings from the saved run configurations.

Setting	CNN baseline	ResNet-50 frozen	ResNet-50 full
Seeds	42	42, 43, 44	42, 43, 44
Batch size	64	32	32
Maximum epochs	20	15	15
Early-stopping patience	5	5	5
Loader workers	6	6	6

Training uses float32 tensors and one optimizer step per batch. The seed function initializes Python, NumPy, and PyTorch generators, requests deterministic cuDNN behavior, and disables cuDNN benchmarking. Loader sampling and workers also receive seeds.

The controlled ResNet study fixes the data, transforms, head learning rate, optimizer, batch size, epoch cap, patience, and paired seeds while varying backbone adaptation.

7.2 Hyperparameter Selection and Configuration Development

The saved draft runs at seed 42 used batch 64 and a 10-epoch cap for frozen ResNet-50, and batch 32, a 5-epoch cap, and one 10^{-4} learning rate for full fine-tuning. Commit `41cf5e3` introduced separate head/backbone optimizer groups and aligned both arms to batch 32, a 15-epoch cap, and head learning rate 10^{-3}. The full backbone retained 10^{-4}. Configuration comments attribute batch 32 to the full model's FP32 memory constraint.

The six final ResNet runs repeat these adopted settings at seeds 42–44. The CNN's saved run uses dropout 0.3. The recorded validation procedure selects checkpoint epochs under each configuration. The separate compute benchmark supplies throughput and memory estimates for planning.

7.3 Hardware and Software

Training records identify an RTX 4060 Laptop GPU, CUDA 12.6, Python 3.10.21, and PyTorch `2.14.1+cu126`. Source commits are `a1246bc` for the CNN and `41cf5e3` for the controlled ResNet runs.

Checkpoint evaluation used Apple MPS, Python 3.13.1, and PyTorch 2.8.0. All seven test runs used batch size 32, zero loader workers, and five CPU threads. Outputs retain device, environment, and checkpoint/configuration/split hashes.

7.4 Checkpoint Selection and Early Stopping

A strict improvement in validation macro-F1 saves `best.pt` and resets patience. Five non-improving epochs or the epoch cap stop training. Table 7 converts saved zero-based indices to one-based epoch numbers.

Table 7: Completed epochs, selected checkpoints, and recorded training duration.

Model	Seed	Completed	Selected epoch	Train time (min)
CNN baseline	42	20	18	20.85
ResNet-50 frozen	42	15	13	13.67
	43	15	11	11.51
	44	15	12	11.23
ResNet-50 full	42	12	7	22.07
	43	14	9	25.61
	44	15	11	27.24

Full seeds 42 and 43 stopped five epochs after their selected checkpoints. The other runs reached their caps. Both ResNet arms share the cap and stopping rule.

7.5 Evaluation Metrics

For N evaluated images with true labels y_i and predictions $\widehat{y}_i$, accuracy is

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}[\widehat{y}_i = y_i]. \tag{3}$$

For class k, let TP_k, FP_k, and FN_k denote true positives, false positives, and false negatives. The primary metric is

$$F1_k = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad \text{Macro-F1} = \frac{1}{30}\sum_{k=0}^{29} F1_k. \tag{4}$$

Macro-F1 includes all 30 classes, assigning zero for a zero denominator. Secondary outputs include per-class precision/recall/F1, confusion matrices, top-3 accuracy, and cross-entropy.

ResNet results use means and sample standard deviations over three seeds. The study averages paired macro-F1 gains and compares them with the observed seed spread, as proposed. The CNN result represents one run.

7.6 Final Test and Compute Protocol

Validation re-evaluation selected illustrative runs before test: CNN seed 42 and both ResNets at seed 44. Each saved checkpoint comes from its original validation history.

All seven checkpoints were tested on 4,984 cleaned images. `results/a2/tables/a2_test_protocol.json` fixes the run selections and checkpoint hashes. Prediction rows follow manifest order.

Training time includes fitting, validation, and checkpoint writes. Forward timing measures one complete test pass after a warm-up batch, synchronizing timed batches. Measured forward time divided by 4,984 gives time per image. Loading, transfers, ranking, and export fall outside this interval. Training uses CUDA and forward timing uses MPS.

Part III
Implementation Results

8 Training Behaviour

The curves reproduce the saved `history.csv` values for the seven runs in Section 7. Epochs start at one. The validation macro-F1 panels mark the selected checkpoint with a circle and the last recorded epoch with a cross.

8.1 CNN Baseline

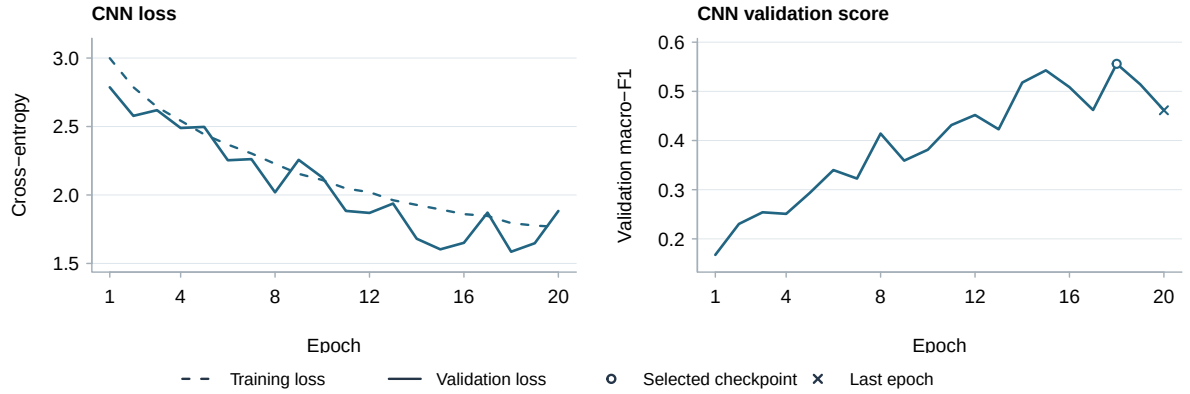


Figure 6: CNN baseline at seed 42. The selected checkpoint is at epoch 18 and the run reaches its 20-epoch cap.

CNN training cross-entropy fell from 2.999 to 1.767 over 20 epochs. Validation macro-F1 rose from 0.167 to a peak of 0.556 at epoch 18, then fell to 0.462 by epoch 20. Validation loss also reached its minimum at epoch 18 (1.586) before rising to 1.883. The run was still reducing training loss at its cap, with substantial validation fluctuations. The single run leaves open how much longer training or a different learning rate would change this pattern.

Training cross-entropy averages augmented batches while parameters are changing. Validation cross-entropy uses fixed parameters and center crops. In addition, dropout is active for the CNN during training. These differences affect the loss gap, so the interpretation uses the trajectories and validation scores together.

8.2 Paired ResNet-50 Runs

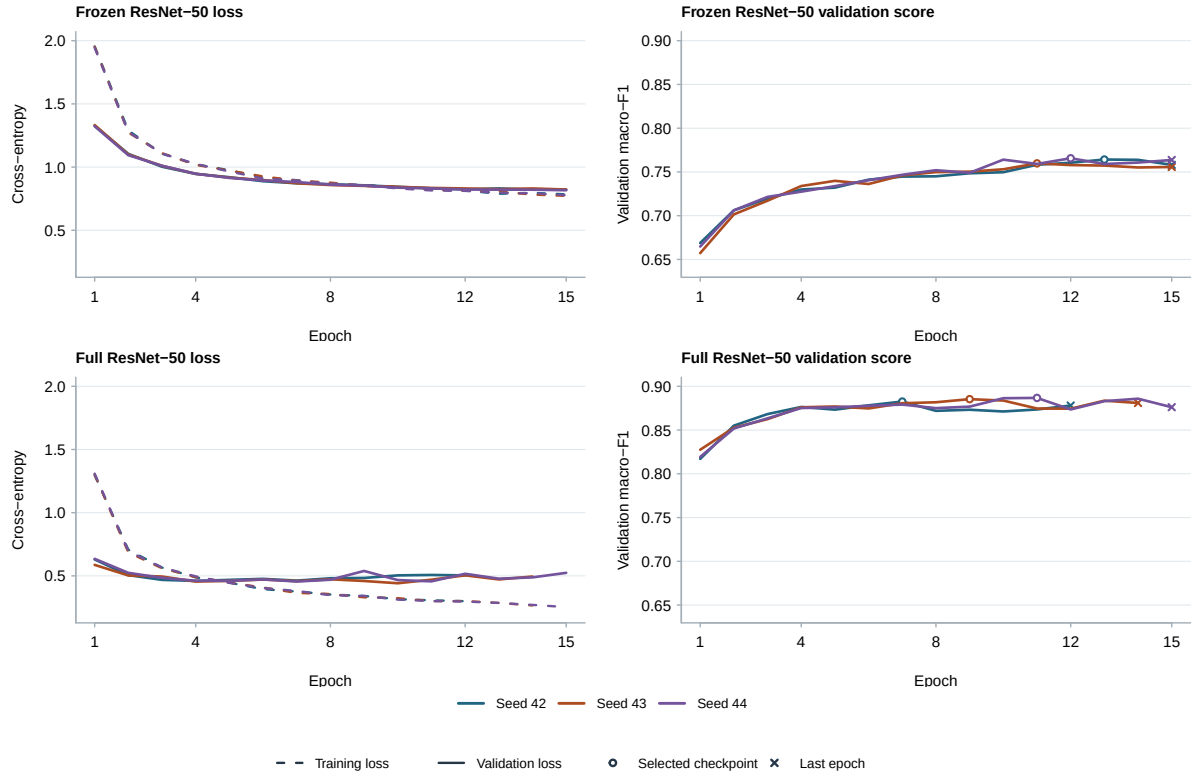


Figure 7: Training loss, validation loss, and validation macro-F1 for the three paired seeds. Both rows use the same axes. Lines end at the last completed epoch.

Frozen backbone. Validation macro-F1 rose from 0.657–0.669 at epoch 1 to peaks of 0.760–0.766 at epochs 11–13. All three runs reached epoch 15, where training loss was 0.772–0.784 and validation loss was 0.817–0.822. Learning slowed late in the run, with the two losses still close.

Full fine-tuning. Validation macro-F1 reached 0.875–0.876 by epoch 4 and peaked at 0.883–0.887. For seed 44, training loss fell from 0.381 at epoch 7 to 0.253 at epoch 15 while validation loss rose from 0.455 to 0.524. This widening gap is consistent with late overfitting under the recorded setup. Seed 44's validation loss was lowest at epoch 7, while macro-F1 selected epoch 11. Loss reflects probabilities and macro-F1 reflects predicted labels, so their optima can occur at different epochs. Seeds 42 and 43 stopped at epochs 12 and 14 after five epochs without improvement. Seed 44 reached the cap.

8.3 Training Time and Implementation

Table 7 records durations including validation. The mean per-run time per completed epoch was 1.04 minutes for CNN, 0.81 for frozen ResNet-50, and 1.83 for full fine-tuning. Full fine-tuning backpropagates through the backbone and updates its parameters. The shared batch of 32 addresses its FP32 memory constraint. The A2 trainer adds separate head and backbone learning rates for full fine-tuning, as described in Section 6.

9 Quantitative Results

9.1 Cleaned Test Performance

All seven saved checkpoints were evaluated on the same 4,984 test images across 30 classes. Table 8 reports accuracy, macro-F1, and top-3 accuracy. The CNN result comes from one draft run at seed 42. Each ResNet-50 row summarizes seeds 42, 43, and 44.

Table 8: Results on the cleaned test split. ResNet-50 cells show the mean and sample standard deviation over three seeds.

Model	Runs	Accuracy	Macro-F1	Top-3 accuracy
CNN baseline	1	0.5451	0.5403	0.7542
ResNet-50 frozen	3	0.7679 ± 0.0013	0.7720 ± 0.0022	0.9169 ± 0.0021
ResNet-50 full	3	0.8908 ± 0.0064	0.8922 ± 0.0063	0.9683 ± 0.0023

Full fine-tuning gives the highest test macro-F1 in every paired seed. The mean paired increase over the frozen backbone is 0.1202 macro-F1, with gains from 0.1150 to 0.1236. The controlled comparison is examined in Section 13. Individual checkpoint scores are available in `results/a2/tables/a2_test_evaluation.csv`.

9.2 Compute Cost

Table 9: Parameter counts and mean costs per run. Peak memory refers to allocated training GPU memory. The dash marks an unrecorded value.

Model	Total parameters	Trainable parameters	Train time (min)	Peak memory (GiB)	Forward (ms/image)
CNN baseline	397,086	397,086	20.85	—	0.859
ResNet-50 frozen	23,569,502	61,470	12.13	0.434	4.312
ResNet-50 full	23,569,502	23,569,502	24.97	2.894	4.285

Training times were recorded on an RTX 4060 Laptop GPU and include validation. Full fine-tuning took about 2.06× the training time and 6.67× the peak allocated GPU memory of the frozen arm. Forward timings use Apple MPS with PyTorch 2.8.0 and batch size 32, following Section 7. Both ResNet variants execute the same architecture at inference and average about 4.3 ms per image under this protocol. The CNN averaged 0.86 ms per image.

9.3 Per-class F1

Figure 8 uses the three checkpoints selected for illustration by validation macro-F1 before test evaluation. These are CNN seed 42 and ResNet-50 seed 44 for both arms.

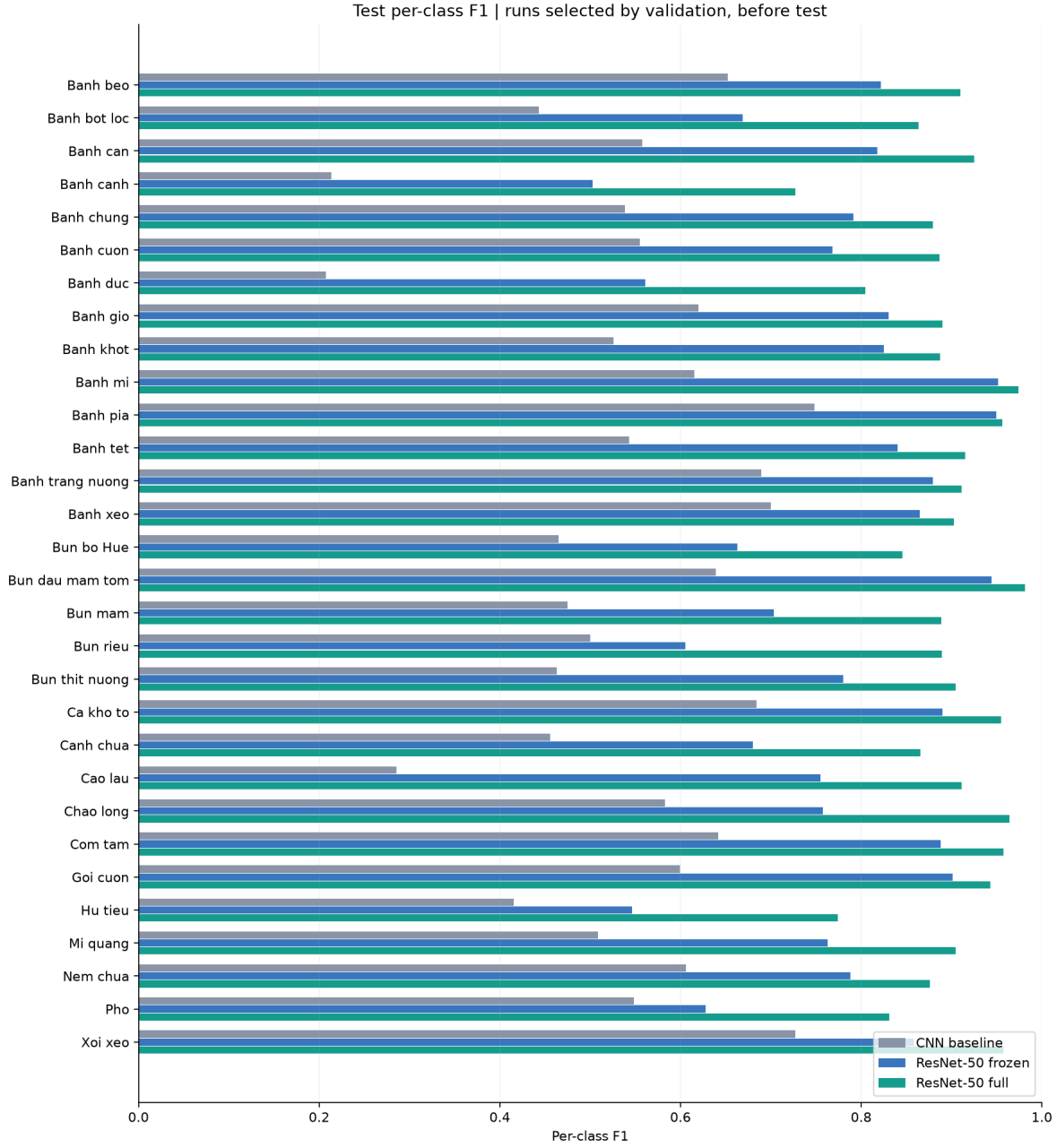


Figure 8: Test F1 by dish class for the validation-selected runs. Each bar represents one checkpoint.

For these runs, full fine-tuning improves F1 over the frozen backbone in all 30 classes. The largest increases are for Bun rieu (0.605 to 0.889), Banh duc (0.561 to 0.805), and Hu tieu (0.546 to 0.774). Banh canh improves from 0.503 to 0.727 and remains the lowest-F1 class for the illustrated full model. Test support varies by class, from 89 images for Banh pia to 302 for Bun bo Hue.

9.4 Confusion Matrix

Figure 9 shows the full fine-tuning checkpoint at seed 44. Each row is normalized by the number of test images in its true class, so the diagonal entries are class recalls.

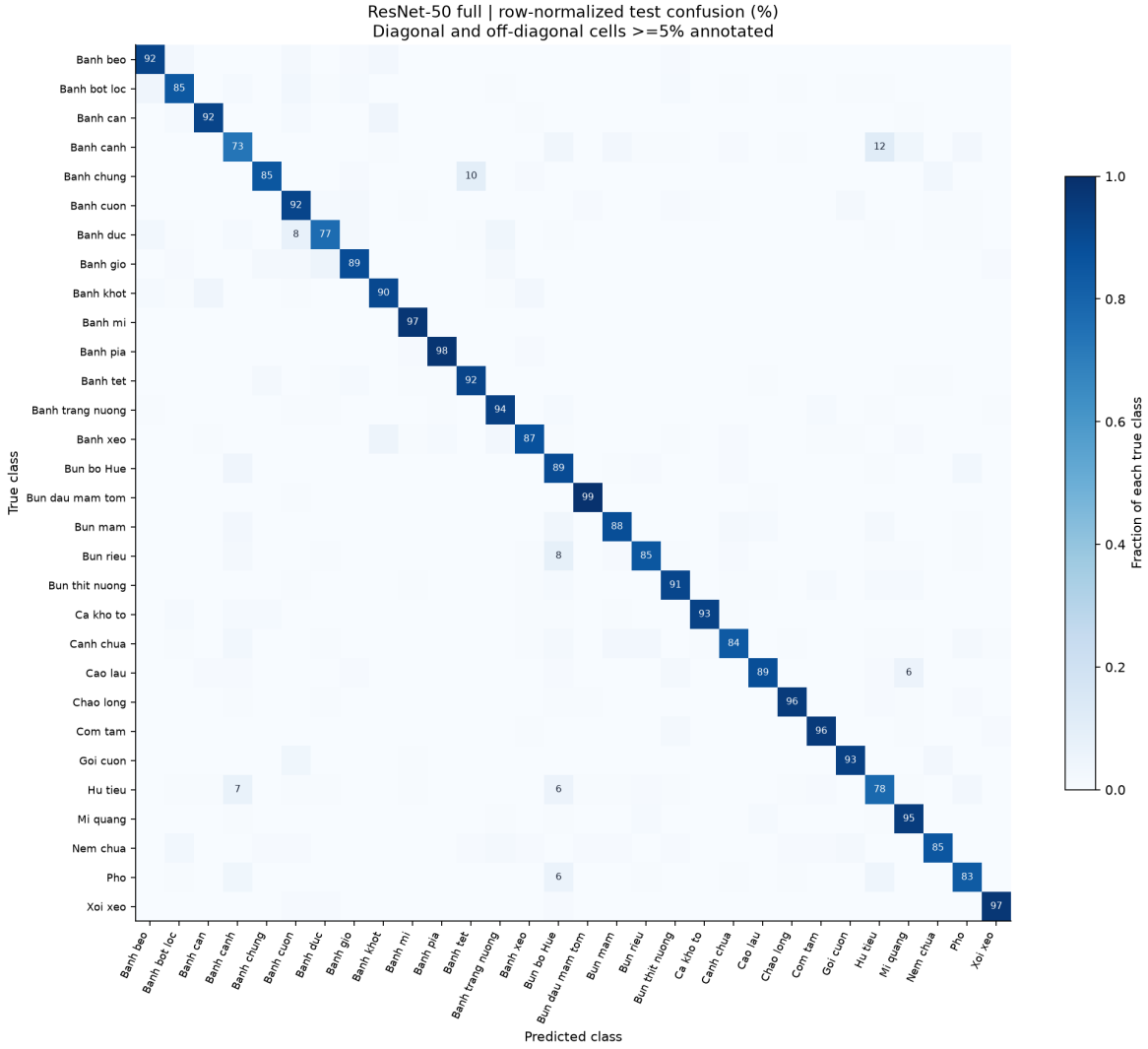


Figure 9: Row-normalized confusion matrix for the validation-selected full fine-tuning checkpoint. Entries annotated in the heatmap are percentages.

Table 10: Selected directed confusions for the three illustrated checkpoints. Each entry is the error count out of the true-class support.

True class	Predicted class	CNN	Frozen	Full
Bun rieu	Bun bo Hue	5/227	52/227	19/227
Banh canh	Hu tieu	37/191	20/191	22/191
Hu tieu	Banh canh	7/194	20/194	13/194

Full fine-tuning reduces the Bun rieu to Bun bo Hue error rate from 22.9% to 8.4% in the illustrated ResNet runs. Banh canh to Hu tieu remains frequent at 11.5%. The full set of confusion matrices for the three illustrated runs is available in `results/a2/figures/test/`.

10 Qualitative Results

The examples use the validation-selected checkpoints from Section 7: CNN seed 42 and both ResNet-50 arms at seed 44. The images show the 224×224 evaluation crops before normalization. The tables give each checkpoint's top-1 label and softmax score.

The galleries prioritize different true classes within each group. Correct examples with a full-model top-1/top-2 score margin of at least 0.5 are ordered by a stable path hash. Hard-correct examples are ranked by their smallest margins. Failures are ranked by directed confusion frequency and then confidence. The paired galleries use path hashing within cases where only one ResNet is correct. The panels below take the first image in each displayed gallery group. The full selection is recorded in `results/a2/error_analysis/qualitative_manifest.csv`.

10.1 Correct, Difficult, and Failed Predictions



Correct (#2250) Difficult (#4165) Failure (#568)

Figure 10: Evaluation crops from the correct, low-margin correct, and failure galleries. IDs index the saved test prediction rows.

Table 11: Predicted labels and top-1 softmax scores for Figure 10.

ID	Dataset label	CNN	Frozen	Full
2250	Bun bo Hue	Bun bo Hue 33.7%	Bun bo Hue 46.8%	Bun bo Hue 99.7%
4165	Goi cuon	Banh bot loc 12.4%	Goi cuon 51.2%	Goi cuon 45.3%
568	Banh canh	Banh duc 40.4%	Hu tieu 80.4%	Hu tieu 99.9998%

The Bun bo Hue crop shows broth, meat, and noodles together, and all three models predict the dataset label. In the Goi cuon example, the dipping-sauce bowl fills much of the crop and only part of the roll is visible. The full model selects Goi cuon over Banh duc by a margin of 0.0047. In the Banh canh example, sliced meats and egg occupy much of the bowl, with white noodles visible around them. Both ResNets predict Hu tieu. The full model assigns nearly all its softmax score to that class.

10.2 Changes After Full Fine-tuning

On the same 4,984 test images, the two illustrated ResNets agree with the dataset label on 3,704 images and both miss 395. The full model alone is correct on 755 images. The frozen model alone is correct on 130.



Full correct (#708) Frozen correct (#4120)

Figure 11: Evaluation crops illustrating changes in both directions between the paired ResNet checkpoints.

Table 12: Predicted labels and top-1 softmax scores for Figure 11.

ID	Dataset label	CNN	Frozen	Full
708	Banh cuon	Banh cuon 26.8%	Banh khot 43.6%	Banh cuon 68.2%
4120	Goi cuon	Goi cuon 78.8%	Goi cuon 99.8%	Banh cuon 79.1%

In #708, herbs and toppings partially cover the small white rolls. The full checkpoint recovers Banh cuon from the frozen checkpoint's Banh khot prediction. In #4120, pale wrappers dominate the crop and the filling is less visible. The full model predicts Banh cuon, while frozen and CNN predict Goi cuon. Wrapper appearance is a plausible source of the ambiguity in this example.

10.3 Grad-CAM for the Banh Canh Failure

Grad-CAM weights final convolutional activations by class-score gradients to produce coarse spatial attributions (Selvaraju et al.). In Figure 12, positive attribution for Hu tieu overlaps parts of the toppings and noodles.

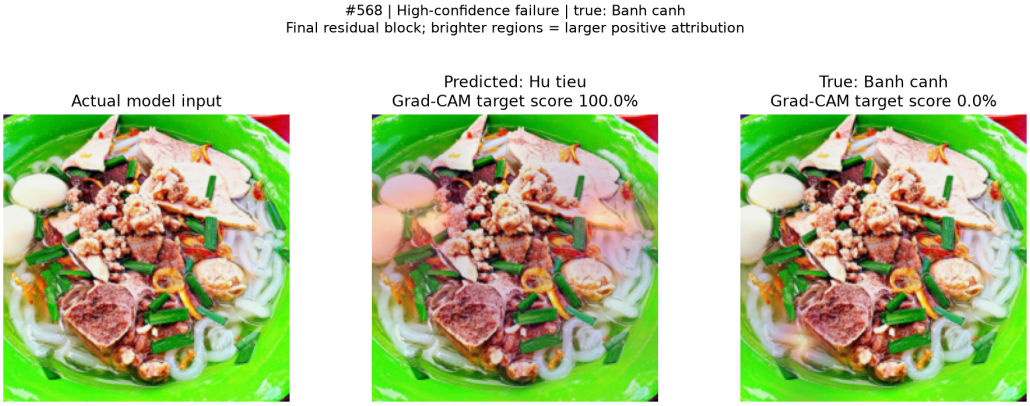


Figure 12: Full ResNet-50, seed 44, on #568. The overlays target the predicted class Hu tieu and the dataset class Banh canh.

11 Comparison and Discussion

11.1 Effect of Backbone Adaptation

Full fine-tuning improves test accuracy by 12.29 percentage points and macro-F1 by 0.1202 over frozen ResNet-50 on average across the paired seeds (Table 8). All three paired gains support the hypothesis that adapting ImageNet features helps on 30VNFoods. Both arms use the same cross-entropy objective, data, classifier architecture, and head learning rate. The gain reflects the combined effect of backbone parameter updates and the associated changes in BatchNorm behavior. The three seeds characterize variation under this setup.

Full fine-tuning reached higher validation macro-F1 early, then showed a widening separation between training and validation loss late in training (Figure 7). The checkpoint rule preserves the epoch with the highest validation macro-F1 within each run. Test performance therefore reflects the selected checkpoints from these learning trajectories.

11.2 Where the Predictions Change

At seed 44, the full checkpoint correctly classifies 625 more test images overall than the frozen checkpoint. Figure 11 illustrates changes in both directions. Bun rieu shows one of the largest classwise improvements. False negatives fall from 102 to 34 and false positives from 61 to 14, so both recall and precision improve.

Banh canh improves from 0.503 to 0.727 F1 as its true positives rise from 90 to 140. The specific Banh canh to Hu tieu confusion nevertheless rises from 20 to 22 of 191 images (Table 10). In the failure crop in Figure 10, meat and egg occupy much of the bowl and white noodles remain visible around them. This case illustrates how shared ingredients and presentation can leave ambiguity even for a class with a large aggregate gain.

11.3 Accuracy and Compute Trade-offs

Full fine-tuning takes roughly twice as long to train as the frozen arm and uses about 6.7 times the peak allocated GPU memory (Table 9). Backpropagation through the backbone accounts for the additional training work. Both variants use the same architecture at inference and average about 4.3 ms per image on the batched MPS protocol. Their main measured cost difference is during training.

The frozen arm gains 22.27 accuracy points over the CNN baseline, while the CNN uses about one fifty-ninth as many parameters and one fifth of the measured forward time. The CNN comparison combines differences in architecture, initialization, normalization, and training budget. Its single saved run provides a reference point for these configurations. The paired ResNet comparison gives the more direct evidence about backbone adaptation.

12 Error Analysis

The full ResNet-50 checkpoint from seed 44, chosen by validation macro-F1, makes 525 errors on 4,984 test images (10.5%). Table 13 counts errors against the eligible images in each group. The groups can overlap, so their counts cannot be added to obtain 525. The examples use the saved prediction IDs from Section 10. They illustrate possible causes, while the counts describe where the model disagrees with dataset labels.

12.1 Error Groups and Examples

Table 13: Diagnostic groups for the validation-selected full ResNet-50. The soup and white-cake families are exploratory. Rates are conditional on the group definitions.

Group	Errors / images	Example and possible cause	Follow-up on validation
Soup family: H2 classes plus Hu tieu and Pho	188 / 1,076 (17.5%)	#568 and #4289 show similar toppings and partly visible noodles.	Review training labels and add clear views of noodle shape. Measure both confusion directions.
Banh chung and Banh tet: errors within the pair	13 / 234 (5.6%)	#678 shows the cut filling with little of the overall shape.	Add labeled whole and cut views. Measure within-pair confusion.
White cakes: Banh duc, Banh cuon, Banh gio, and Banh bot loc	85 / 624 (13.6%)	In #978, pale folds and minced toppings resemble the predicted Banh gio.	Review ambiguous training labels and add examples with clear shapes.
Extreme aspect ratio: width/height below 0.5 or above 2	4 / 52 (7.7%)	In #191, the crop excludes much of the tray and condiments.	Compare padding to a square with center cropping under matched training settings.
Top-1 softmax score at least 0.9	120 / 4,005 (3.0%)	#568 assigns nearly all probability to a visually similar dish.	Measure calibration and error rates at candidate review thresholds.
Top-1 minus top-2 score at most 0.1	68 / 109 (62.4%)	In #1669, small, blurred pastries compete with the tea setting.	Measure error versus review workload when flagging low-margin cases.

The Banh canh–Hu tieu confusion is asymmetric. Banh canh is predicted as Hu tieu in 22 of 191 cases (11.5%), and Hu tieu is predicted as Banh canh in 13 of 194 (6.7%). The confidence groups also answer different questions. Only 3.0% of predictions with a score of at least 0.9 are wrong, but those 120 predictions account for 22.9% of all errors.

12.2 Checks of the EDA Hypotheses

H1 (rice-cake shape). Of 234 true Banh chung or Banh tet images, 13 (5.6%) are confused with the other class, accounting for 13 of the 26 errors on the pair. Banh chung is predicted as Banh tet 10 times out of 99, versus 3 out of 135 in reverse. The cut view in #678 illustrates a missing shape cue. An annotated comparison of whole and cut views would test whether that cue explains the pattern across the pair.

H2 (similar soup dishes). Of 720 true Banh canh, Bun bo Hue, or Bun rieu images, 46 (6.4%) are confused with another class in that group, accounting for 46 of its 118 errors. Bun rieu is predicted as Bun bo Hue in 19 of 227 cases. The reddish broth and toppings in #2973 are consistent with visual overlap. Annotations of broth and noodle visibility would help test which cues explain these confusions.

H3 (extreme aspect ratios). The extreme group has 4 errors in 52 images (7.7%, 95% Wilson interval 3.0–18.2%), compared with 521 in 4,932 (10.6%) for the other images. Applying the other images' error rates within each true class gives 4.60 expected errors among the 52 extreme images. The observed rates do not support an increase for this run. The small group leaves substantial uncertainty. Figure 13 illustrates how cropping can remove dish context in an individual case.



Figure 13: Original image and evaluation crop for #191. The crop retains the center of the tray, while much of the serving context falls outside it.

H4 (Banh pia background and packaging). The white-border proxy requires at least half the pixels in the outer 13-pixel band of a 64×64 thumbnail to have all RGB channels at least 220 and a channel range of at most 30 on a 0–255 scale. It includes white plates and food. The model makes 0 errors on 11 Banh pia images meeting the proxy and 2 on the other 78 (2.6%). Grad-CAM overlaps pastry in the correct #1623 case and part of the tea setting for the wrong prediction on #1669. These observations leave the shortcut hypothesis unresolved. Masking annotated background and packaging regions while preserving the food on validation would provide a more direct test.

Group counts and image IDs are available in `results/a2/tables/a2_test_error_groups.csv` and `results/a2/error_analysis/qualitative_review.csv`. These diagnostics describe a single checkpoint. The proposed follow-ups are future validation experiments.

13 Ablation Study

13.1 Controlled Factor and Hypothesis

We compare a fixed ImageNet ResNet-50 backbone with one adapted to 30VNFoods. We expect full fine-tuning to improve macro-F1 because the visual cues that distinguish Vietnamese dishes can differ from those learned on ImageNet. In the frozen arm, only the classifier head is optimized. Backbone gradients are disabled and BatchNorm stays in evaluation mode. In the full arm, the backbone receives optimizer updates at 10^{-4} and its BatchNorm running statistics update during training. The experiment measures the combined effect of these backbone changes.

Both arms start from the same pretrained weights and use the same classifier architecture, cleaned splits, 224×224 inputs, ImageNet normalization, and training augmentation. They share cross-entropy, Adam, weight decay of 10^{-4}, a head learning rate of 10^{-3}, and batch size 32. For each of seeds 42, 43, and 44, both arms have a 15-epoch cap and stop after five epochs without improvement in validation macro-F1. The resulting run lengths can differ under that shared stopping rule (Table 7). Each run is evaluated on the same test images using the checkpoint with the highest validation macro-F1.

13.2 Paired Test Results

Table 14: Frozen and full test results by matched seed. Deltas are full minus frozen, computed before rounding. Accuracy deltas are in percentage points (pp).

	Accuracy			Macro-F1 (primary)		
Seed	Frozen	Full	Δ (pp)	Frozen	Full	Δ
42	76.67%	88.34%	+11.68	0.7698	0.8849	+0.1150
43	76.77%	89.43%	+12.66	0.7720	0.8957	+0.1236
44	76.93%	89.47%	+12.54	0.7742	0.8961	+0.1219
Mean	76.79%	89.08%	+12.29	0.7720	0.8922	+0.1202

The proposal requires a gain larger than the observed spread across three seeds. We quantify spread as the maximum minus minimum test macro-F1 within each arm. The ranges are 0.0044 for frozen and 0.0112 for full. Even the smallest paired gain, 0.1150, exceeds the larger range. The result meets the proposal criterion on these runs. This is a descriptive comparison across three seeds.

Full fine-tuning raises mean training time from 12.13 to 24.97 minutes and mean peak allocated training GPU memory from 0.43 to 2.89 GiB, with similar measured forward time (Table 9). The paired scores are saved in `results/a2/tables/a2_test_paired_seed_gains.csv`.

14 Limitations

The evaluation covers one cleaned split of 30VNFoods. Hash-based filtering reduces duplicate leakage (Section 3), and near duplicates outside the chosen thresholds can remain. A spot check of the drop list also found distinct Banh canh and Bun mam photos grouped with retained images of the same class at Hamming distance eight. Distinct images from the same website can appear in multiple splits because source-domain separation was not verified. The web collection may overrepresent certain plating styles and backgrounds, and folder labels may remain ambiguous. Generalization to new sources or user photos remains unmeasured. The proposal's comparison with the original 5,040-image test set remains pending. The reported scores cover the 4,984-image cleaned test set.

The ResNet-50 results span three seeds on the same split. The observed spread describes variability under this setup. The CNN baseline has one saved run, so its variability across seeds is unknown. Its architecture and training settings differ from the ResNet-50 arms, which limits what that comparison can attribute to pretraining. The paired ResNet study estimates the joint effect of updating backbone parameters and BatchNorm statistics (Section 13).

The error analysis uses one validation-selected full checkpoint. The extreme-aspect group has 52 images and the white-border Banh pia group has 11, so their error rates are imprecise. The crop examples and Grad-CAM maps help identify regions to inspect, while the white-border proxy can include plates and food. To test the suspected mechanisms, future validation runs could compare center crops with aspect-preserving padding under matched training settings and measure prediction changes when annotated background or packaging is masked while the food stays visible.

Training times include validation on an RTX 4060 Laptop GPU. Forward times come from one batched Apple MPS pass per checkpoint and exclude image loading, preprocessing, and transfers. They describe model forward cost under that protocol. Estimating deployment latency would require repeated end-to-end measurements on the intended hardware and batch size.

15 Conclusion

Full fine-tuning yields the best test performance on the cleaned 30VNFoods split. Mean accuracy rises from 76.79% with a frozen ResNet-50 backbone to 89.08% with full adaptation, and mean macro-F1 rises from 0.7720 to 0.8922. All three paired seeds improve, with a mean gain of 12.29 percentage points in accuracy and 0.1202 in macro-F1. The scratch CNN provides a smaller and faster reference under the measured forward protocol, with 54.51% accuracy and 0.5403 macro-F1 from one run.

Adapting the backbone increases mean training time from 12.13 to 24.97 minutes and peak allocated training GPU memory by about 6.7× relative to freezing. Both ResNet variants average about 4.3 ms per image in the measured batched forward pass. In this setup, the higher predictive quality of full fine-tuning comes mainly at a training cost.

The Banh canh to Hu tieu confusion persists after fine-tuning. A next experiment would compare the current center crop with aspect-preserving padding under matched training settings. On validation, we would record noodle visibility in both input views and compare the two directions of this confusion alongside overall macro-F1.

16 Reproducibility

16.1 Environment, Dataset, and Checkpoints

Use the group repository and run the commands from its root. The batch evaluator requires Python 3.11 or later. The recorded Apple MPS evaluation used Python 3.13.1 and PyTorch 2.8.0. The test protocol uses batch size 32, five CPU threads, and zero loader workers. The training logs record an NVIDIA GeForce RTX 4060 Laptop GPU, CUDA 12.6, Python 3.10.21, and PyTorch `2.14.1+cu126`.

Download Kaggle dataset `quandang/vietnamese-foods/versions/11` and place the extracted `Images/` directory at `data/30VNFoods/Images/`. All seven runs share the same image paths, labels, and class order in `results/a2/splits/split_v2.json`. The EDA notebook `notebooks/a2_eda.ipynb` documents how the manifest was generated.

Obtain the saved weights from the group checkpoint folder. Place each model's weights at `results/a2/runs/<run_id>/best.pt`, beside that run's saved `config.yaml`. The batch evaluator discovers every `best.pt` under that directory. Use the seven checkpoints in Table 15 to reproduce the reported aggregates.

Table 15: Reported runs, training commits, and selected epochs. Epoch numbers are one-based.

Run ID	Seed	Best epoch	Training commit
`a2_cnn_s42_1028-1546`	42	18	`a1246bc`
`a2_resnet50_frozen_s42_1110-0659`	42	13	`41cf5e3`
`a2_resnet50_frozen_s43_1110-0910`	43	11	`41cf5e3`
`a2_resnet50_frozen_s44_1110-1151`	44	12	`41cf5e3`
`a2_resnet50_full_s42_1110-0713`	42	7	`41cf5e3`
`a2_resnet50_full_s43_1110-1121`	43	9	`41cf5e3`
`a2_resnet50_full_s44_1110-1204`	44	11	`41cf5e3`

Each run directory contains its resolved configuration, epoch history, and training metrics. Its `evaluation/test/metrics.json` records the checkpoint, config, and split SHA-256 hashes alongside the training and evaluation commit fields. The test logs identify the evaluation worktree as `a7bd85f-dirty`. The committed evaluator and exports are available at `ee975d4`. The dirty marker limits exact source reconstruction for the original evaluation. The checkpoint hashes for all seven runs also appear in `results/a2/tables/a2_test_protocol.json`.

16.2 Rerun Evaluation and Error Analysis

In a virtual environment, select PyTorch and Torchvision builds for the target device, then install the remaining dependencies. Keep the saved validation exports in each run directory. The batch evaluator uses those scores to select illustrative checkpoints and checks that the selection agrees with the saved protocol.

```
python -m pip install -r requirements.txt
python scripts/a2_evaluate_all.py --split test --device mps --batch-size 32
python scripts/a2_error_analysis.py
python scripts/a2_gradcam.py
python scripts/a2_error_analysis.py
```

The evaluator reuses complete exports when their hashes and evaluation settings match. To recompute predictions or use a different device, run the sequence in a separate copy with `--rerun` and the corresponding `--device` value on the evaluator command. Forward timings depend on the device. The first analysis pass prepares error groups and example selections, Grad-CAM produces attribution figures on CPU, and the second pass incorporates those figures and the saved visual review.

Per-run predictions and confusion matrices are stored under each run's `evaluation/test/` directory. Table 8 uses `results/a2/tables/a2_test_summary.json`, and Table 14 uses the per-run test export `results/a2/tables/a2_test_evaluation.csv`. The analysis writes subgroup counts to `results/a2/tables/a2_test_error_groups.csv` and figures to `results/a2/figures/test/`.

16.3 Rebuild Checkpoints from Training

To reconstruct weights, use a separate checkout of the training commit in Table 15 with the same dataset layout and split. Train the CNN at `a1246bc` with the original 20-epoch override. Train both ResNet arms at `41cf5e3`, repeating the ResNet commands for seeds 43 and 44.

```
python -m src.a2.train --model cnn --seed 42 --epochs 20
python -m src.a2.train --model resnet50_frozen --seed 42
python -m src.a2.train --model resnet50_full --seed 42
```

The historical base configs and these arguments resolve to the saved settings. The trainer writes a new run directory and selects `best.pt` by validation macro-F1. Use the original weights and run IDs to reproduce the exact checkpoint fingerprints.

AI Usage Disclosure

OpenAI Codex. Nguyễn Nhật Nam used Codex to work on the A2 report one section at a time. For each section, Nam selected the scope and worked with Codex to check the relevant source code, experiment logs, or prediction exports. Codex assisted with the LaTeX draft and explanations for that section. Nam reviewed the draft, requested revisions as needed, and decided when to move on. Codex began work on the next section only after Nam had reviewed the current one and asked to continue. Codex also assisted with checkpoint evaluation and error analysis, which supplied evidence for the report. A representative prompt asked for help checking the saved results before drafting the current section. Nam is responsible for final verification.

Claude Code (Opus 5.5). Hồ Hồng Phúc Nguyên used Claude Code for the October dataset proposal and A2 draft. It helped interpret the handbook, review the EDA, draft the proposal, implement training code, and document the assignment page. A representative prompt asked for a review of the EDA and a proposal revision based on the work already completed. Nguyên checked the split against the local images, tested data loading and model forward passes, and checked the reported numbers against the EDA and run outputs. Nguyên is responsible for final verification of these materials.

The repository's `AI_USAGE.md` records the affected files, representative prompts, and verification sources.

References

- CO3133 course handbook, Sections 3, 4, and 18–22.
- 30VNFoods (Vietnamese Foods), Kaggle version 11. Dataset release.
- *30VNFoods: A Dataset for Vietnamese Foods Recognition*, IEEE COMNETSAT, 2021. Publication.
- K. He, X. Zhang, S. Ren, and J. Sun. *Deep Residual Learning for Image Recognition.* CVPR, 2016. Paper.
- Torchvision. *ResNet-50 model and ImageNet-1K V2 weights.* Model documentation.
- PyTorch. CrossEntropyLoss and Adam API documentation.
- R. R. Selvaraju et al. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* ICCV, 2017. Paper.

A Dataset Proposal and Approval

The group submitted the dataset proposal on 7 October 2026 and recorded a revision dated 28 October 2026 in `reports/a2/proposal.md`. The approval record in `reports/a2/approval.md` documents the submission and revision. The instructor's decision, decision date, and any conditions remain unverified in the available records.

B Supplementary Evidence

Table 16 gives the test scores for each checkpoint on the cleaned split. Match each model and seed to its run ID in Table 15. The per-image predictions, per-class metrics, and confusion matrix for each run are stored in its `evaluation/test/` directory. The complete figures are in `results/a2/figures/test/`, and the example IDs and selection rules are in `results/a2/error_analysis/qualitative_manifest.csv`.

Table 16: Individual checkpoint scores on the cleaned test split.

Model	Seed	Accuracy	Macro-F1	Weighted F1	Top-3 accuracy
CNN from scratch	42	0.5451	0.5403	0.5372	0.7542
ResNet-50 frozen	42	0.7667	0.7698	0.7646	0.9147
ResNet-50 frozen	43	0.7677	0.7720	0.7664	0.9171
ResNet-50 frozen	44	0.7693	0.7742	0.7685	0.9189
ResNet-50 full	42	0.8834	0.8849	0.8837	0.9665
ResNet-50 full	43	0.8943	0.8957	0.8939	0.9675
ResNet-50 full	44	0.8947	0.8961	0.8947	0.9709

C Team Contributions

Table 17: Responsibilities and deliverables for Assignment 2.

Member	Responsibility and deliverables
Nguyễn Nhật Nam 2352778	**Evaluation and report.** Implemented the checkpoint evaluator, ran the cleaned test comparison, and prepared the qualitative examples and error analysis. Led the section-by-section development of the LaTeX report and assembled its tables and figures.
Hồ Hồng Phúc Nguyên 2352824	**Data and training.** Prepared the dataset proposal, conducted EDA, and built the cleaned split. Implemented the data and training pipeline, trained the scratch CNN, and ran the paired frozen and full ResNet-50 experiments.
Phan Quốc Đại Sơn 2353053	**Presentation.** Led preparation of the final presentation deck, organizing the dataset, methodology, experiment results, and error analysis for the oral presentation.